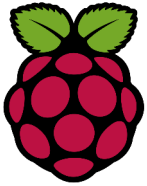


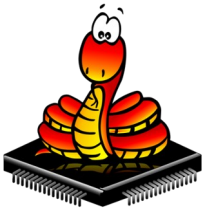


Atelier Raspi

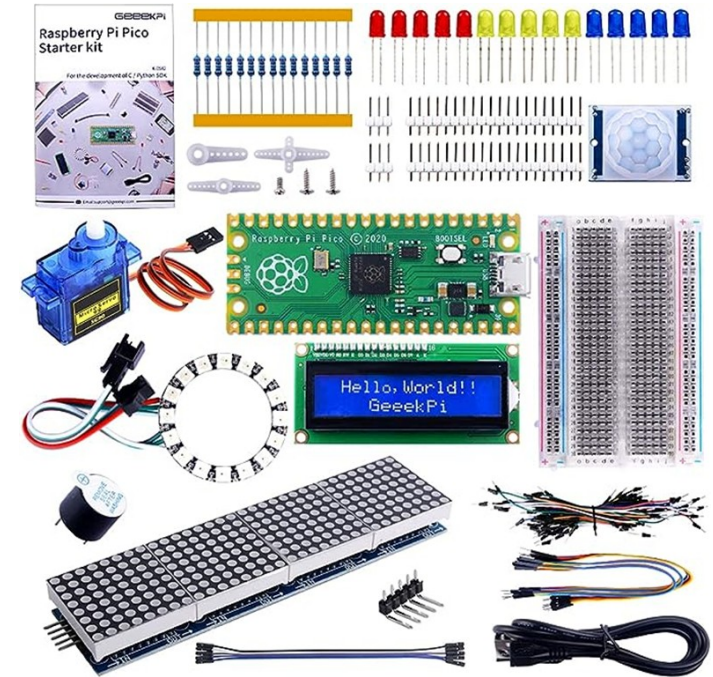
Atelier N°19 Le Jeu de Simon



Logo du Raspberry Pico



Logo du MicroPython



L'atelier a pour valeurs, le partage, l'aide, la formation, le faire et construire ensemble à partir de l'expérience des participants

Atelier N°19 Le Jeu de Simon

- Etape 1 Le jeu classique**
- Etape 2 Rien ne se jette....**
- Etape 3 Rappel du câblage**
- Etape 4 Un programme évolutif !**
- Etape 5 Code de base (1/2)**
- Etape 6 Code de base (2/2)**
- Etape 7 Code LCD à réutiliser**
- Etape 8 Code Neopixel à réutiliser (1/2)**
- Etape 9 Code Neopixel à réutiliser (2/2)**



A19E1 Le jeu classique

JEU SIMON (Classique)

LE JEU ÉLECTRONIQUE DE MÉMOIRE LUMINEUSE ET SONORE



VUE DE DESSUS (FACE AVANT)

FONCTIONNEMENT

1. Le jeu génère une séquence de couleurs et de sons.
2. Le joueur doit répéter la séquence en appuyant sur les boutons dans le bon ordre.
3. À chaque niveau réussi, la séquence s'allonge.
4. Le jeu s'arrête en cas d'erreur (Game Over).
5. Le score (niveau atteint) est affiché.

CARACTÉRISTIQUES

- 4 BOUTONS LUMINEUX (LED)
 - – ROUGE
 - – VERTE
 - – JAUNE
 - – BLEUE
- MODES DE JEU : 1, 2, 3
- AFFICHAGE SCORE (2 CHIFFRES)
- SONORISATION (BIPS)
- FONCTION "LONGEST" (MEILLEUR SCORE)
- ALIMENTATION : 3 PILES LR14 (C)
- CONSTRUCTION ROBUSTE EN PLASTIQUE ABS

A19E2 Rien ne se jette, rien ne s'achète, tout se réutilise !

JEU SIMON (Classique)

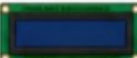
LE JEU ÉLECTRONIQUE DE MÉMOIRE LUMINEUSE ET SONORE

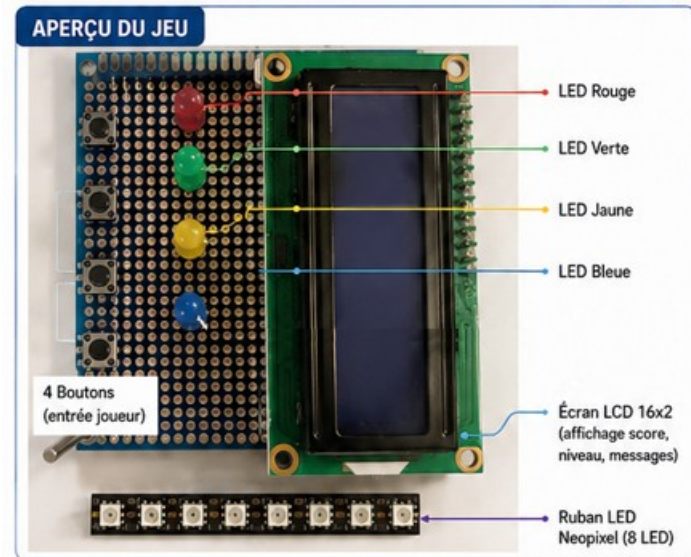


VUE DE DESSUS (FACE AVANT)



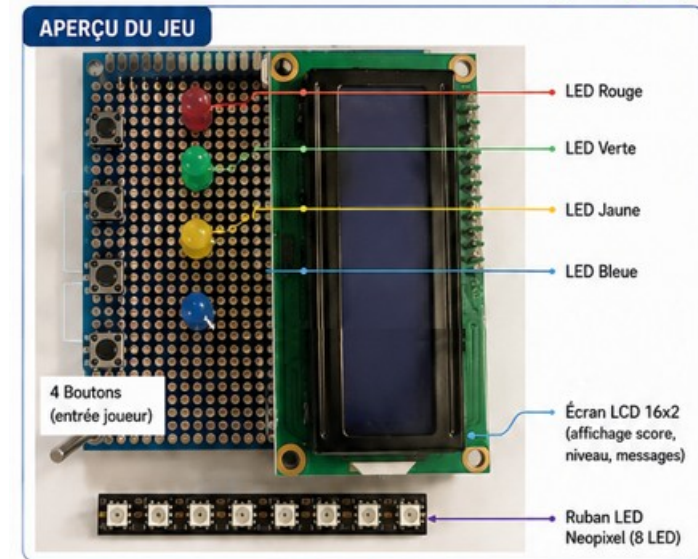
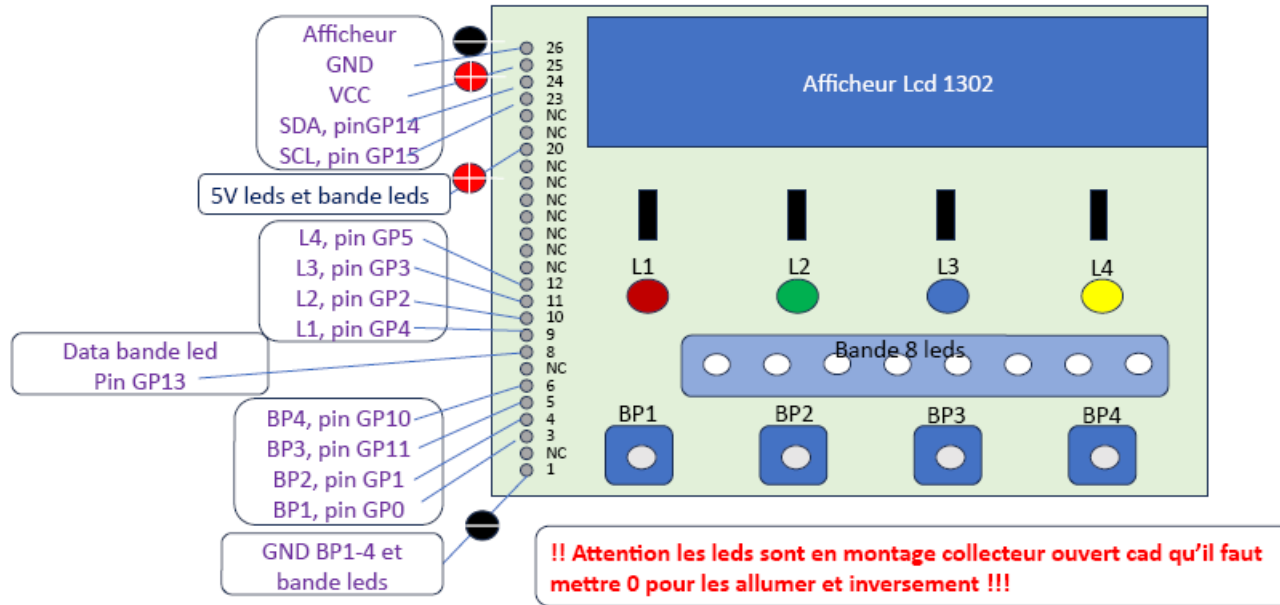
COMPOSANTS UTILISÉS

-  - 1 × Raspberry Pi Pico
-  4 × LED 5mm (Rouge, Verte, Jaune, Bleue)
-  4 × Boutons poussoirs (entrée joueur)
-  1 × Ruban LED Neopixel (8 LED - WS2812B)
-  1 × Écran LCD 16×2 (I2C)
-  Résistances 220 Ω (LED)



A19E3 Rappel du câblage

Module interface homme machine



A19E4 Un programme évolutif !

```
#Version 1: Affichage des informations du jeu sur la console
#
#Version 2: Affichage des informations du jeu sur un ecran LCD
#
#Version 3: Utilisation du ruban Neopixel pour afficher le niveau de jeu par exemple
# Niveau 1 à 4: toutes les leds en vert
# Niveau 5 à 10: "" en jaune
# Niveau 10 et +: "" en rouge
#
#Version 4: Utilisation du ruban Neopixel comme bar graph...
# Niveau 1 à 4: leds 0-2 en vert
# Niveau 5 à 10: leds 0-2 en vert + leds 3-5 en jaune
# Niveau 10 et +: leds 0-2 en vert + leds 3-5 en jaune + leds 6-8 en rouge
#
#Pour aller plus loin:
# Faire clignoter toutes les leds quand on perd
# Faire une animation quand on gagne
# Accélérer le jeu après le niveau 5
```


1. MATÉRIEL

LEDs (sorties)



Rouge
Pin 4



Verte
Pin 2



Bleue
Pin 3



Jaune
Pin 5

Boutons (entrées, pull-up)



Bouton
Rouge
Pin 0



Bouton
Vert
Pin 1



Bouton
Bleu
Pin 11



Bouton
Jaune
Pin 10

2. FONCTIONS UTILES

Commande des LEDs (collecteur ouvert)

led_on(i) = allume la LED i

led_off(i) = éteint la LED i

allumer_led(i)

1. Allume LED i pendant 0.5 s
2. Éteint puis attend 0.2 s

attendre_bouton()

- Attend l'appui sur un bouton
- Allume la LED correspondante
- Attend le relâchement
- Retourne l'index du bouton (0 à 3)

afficher_sequence()

- Attend 0.5 s
- Allume les LEDs de la séquence une par une

erreur()

- Signale une erreur (ex : clignotement rapide des 4 LEDs)
- Attend un moment

victoire()

- Signale une bonne réponse (ex : les 4 LEDs clignotent)
- Court délai

3. PROGRAMME PRINCIPAL (BOUCLE INFINIE)

INITIALISATION

- Éteindre toutes les LEDs
- Les allumer pour test
- Les éteindre
- sequence_jeu = [] (liste vide)

DÉBUT D'UNE NOUVELLE MANCHE

- Ajouter une nouvelle couleur aléatoire à la fin de sequence_jeu (0 à 3)
- niveau = longueur de la séquence
- Afficher le niveau

AFFICHER LA SÉQUENCE

- afficher_sequence()
- Le jeu montre la suite de couleurs

TOUR DU JOUEUR

- Pour chaque couleur attendue dans sequence_jeu :
- choix_joueur = attendre_bouton()
 - Le joueur doit reproduire la suite

Le choix du joueur
est-il correct ?

NON

ERREUR

- Afficher "Perdu ! Score : niveau-1"
- erreur()
- sequence_jeu = []
(on recommence depuis le début)

BONNE RÉPONSE

- victoire()
- Attendre un court instant

4. RÈGLE DU JEU

Le jeu ajoute une couleur aléatoire à chaque manche.

Le joueur doit reproduire toute la séquence dans le même ordre.

Si le joueur se trompe → le jeu s'arrête et recommence.

S'il réussit → une nouvelle couleur est ajoutée et ça continue !

5. EXEMPLE DE PARTIE

Séquence affichée par le jeu



Séquence affichée par le jeu



Séquence affichée par le jeu



...

Le joueur doit toujours répéter toute la séquence.

LÉGENDE

Fonctions / blocs



Actions du jeu



Affichage / attente



Décision



Erreur



Succès



Flux du programme



A19E5 Code de base

```
1 from machine import Pin
2 import time
3 import random
```

```
24 #-----
25 # Declaration des boutons et des leds
26 #-----
27 |
28 # Definition des LEDs
29 leds = [
30     Pin(4, Pin.OUT), #rouge
31     Pin(2, Pin.OUT), #verte
32     Pin(3, Pin.OUT), #bleu
33     Pin(5, Pin.OUT) #jaune
34 ]
35
36 # Definition des boutons
37 boutons = [
38     Pin(0, Pin.IN, Pin.PULL_UP), #bouton sous la led rouge
39     Pin(1, Pin.IN, Pin.PULL_UP), #bouton sous la led verte
40     Pin(11, Pin.IN, Pin.PULL_UP), #bouton sous la led bleu
41     Pin(10, Pin.IN, Pin.PULL_UP) #bouton sous la led jaune
42 ]
```

```
44 #-----
45 # Fonctions utiles pour simplifier la lisibilité du code
46 #-----
47
48 # Comme les leds sont branchées en "collecteur ouvert" leur commande est "inversée"
49 # On définit 2 fonctions "pour le cacher"
50 def led_on(index):
51     leds[index].off()
52
53 def led_off(index):
54     leds[index].on()
55
56 # Pour simplifier la lecture du code on regroupe toutes les commandes
57 # pour allumer la led pendant un instant dans une fonction
58 def allumer_led(index):
59     led_on(index)
60     time.sleep(0.5)
61     led_off(index)
62     time.sleep(0.2)
63
```

```
64 #-----
65 # Fonctions du jeu
66 #-----
67
68 # On stocke également la gestion du bouton dans une fonction
69 # pour simplifier la lecture du code
70 def attendre_bouton():
71     while True:
72         for i, bouton in enumerate(boutons):
73             if bouton.value() == 0: # appui
74                 led_on(i)
75                 while bouton.value() == 0:
76                     pass
77                 led_off(i)
78                 time.sleep(0.1)
79                 return i
80
81 def afficher_sequence():
82     time.sleep(0.5)
83     for couleur in sequence_jeu:
84         allumer_led(couleur)
```


A19E6 Code de base

```
86 #-----
87 # Fonctions vides à compléter !!!!
88 #-----
89
90 def erreur():
91     # pass indique qu'on ne fait rien dans cette fonction pour l'instant
92     # et que c'est ok
93     pass
94
95 def victoire():
96     # pass indique qu'on ne fait rien dans cette fonction pour l'instant
97     # et que c'est ok
98     pass
99
```

```
100 #-----
101 # Initialisation
102 #-----
103
104 # On éteint toutes les leds
105 for i in range(4):
106     led_off(i)
107 time.sleep(1)
108
109 # On les rallume pour vérifier qu'elles sont toutes ok
110 for i in range(4):
111     led_on(i)
112 time.sleep(1)
113
114 # On les re éteint pour commencer à jouer
115 for i in range(4):
116     led_off(i)
117 time.sleep(1)
118
119 # On vide la liste qui contient la sequence de jeu
120 # On va la construire pas à pas dans le code
121 sequence_jeu = []
122
```

```
123 #-----
124 # Programme principal
125 #-----
126
127 print("Jeu de Simon")
128
129 while True:
130     # Ajoute une nouvelle étape
131     sequence_jeu.append(random.randint(0, 3))
132
133     # Calcul du niveau selon la longueur de la sequence
134     niveau = len(sequence_jeu)
135     print("Niveau :", niveau)
136
137     # On joue la séquence
138     afficher_sequence()
139
140     # Vérification de la saisie du joueur
141     perdu = False
142     for couleur_attendue in sequence_jeu:
143         choix_joueur = attendre_bouton()
144         if choix_joueur != couleur_attendue:
145             print("Perdu ! Score :", len(sequence_jeu) - 1)
146             erreur()
147             sequence_jeu = [] #on recommence, on efface la sequence de jeu
148             break
149         else:
150             #print("Bravo !")
151             victoire()
152
153     # On attend un peu
154     time.sleep(1)
```

A19E7 Code LCD à réutiliser

```
1 from machine import Pin, I2C
2 from machine_i2c_lcd import I2cLcd
3 import time
4
5 # Declaration de l'ecran LCD
6 i2c = I2C(1, scl=Pin(15), sda=Pin(14), freq=400000)
7 addr = i2c.scan()[0]
8 #print(hex(addr))
9 lcd = I2cLcd(i2c, addr, 2, 16)
10
11
12 while True:
13     lcd.clear()
14     lcd.putstr(" Hello Raspi ! \n")
15     lcd.putstr(" MJC 2025-2026 \n")
16     time.sleep(1)
17
```

A19E8 Code Neopixel à réutiliser (1/2)

```
1 from neopixel import Neopixel
2 import time
3
4 # Declaration du ruban de leds
5 NUM_LED = 8
6 PIN_NB = 13
7 ruban = Neopixel(NUM_LED, 0, PIN_NB, "GRB")
8
9 # on definit la luminosite des leds / 255
10 ruban.brightness(15)
11 # on eteint toute les leds
12 ruban.clear()
13 # on affiche
14 ruban.show()
15
16
17 while True:
18     # on allume toutes les leds du sapin en rouge pendant 1 sec
19     ruban.fill((255,0,0))
20     ruban.show()
21     time.sleep(1)
22     # on allume toutes les leds du sapin en vert pendant 1 sec
23     ruban.fill((0,255,0))
24     ruban.show()
25     time.sleep(1)
26     # on allume toutes les leds du sapin en jaune pendant 1 sec
27     ruban.fill((255,255,0))
28     ruban.show()
29     time.sleep(1)
30 |
```

A19E9 Code Neopixel à réutiliser (2/2)

```
1 from neopixel import Neopixel
2 import time
3
4 # Declaration du ruban de leds
5 NUM_LED = 8
6 PIN_NB = 13
7 ruban = Neopixel(NUM_LED, 0, PIN_NB, "GRB")
8
9 # Declaration de quelques couleurs
10 couleur_rouge = (255,0,0)
11 couleur_vert = (0,255,0)
12 couleur_bleu = (0,0,255)
13 couleur_noir = (0,0,0)
14 couleur_jaune = (255,255,0)
15
16 # on definit la luminosite des leds
17 ruban.brightness(15)
18 # on eteint toute les leds
19 ruban.clear()
20 # on affiche
21 ruban.show()
```

```
23 while True:
24     # on allume tour a tour les leds du sapin en bleu
25     for i in range (0,NUM_LED):
26         ruban.set_pixel(i,couleur_bleu)
27         ruban.show()
28         time.sleep(0.1)
29     ruban.clear()
30     ruban.show()
31
32     # on allume un bloc de leds
33     ruban.set_pixel_line(5,7,couleur_rouge)
34     ruban.show()
35     time.sleep(2)
36     ruban.clear()
37     ruban.show()
38
39     # on fait un gradient de couleur
40     ruban.set_pixel_line_gradient(1,7,couleur_vert,couleur_bleu)
41     ruban.show()
42     time.sleep(2)
43     ruban.clear()
44     ruban.show()
```